FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of 12 JULY 2021 - Part Theory

LAST + FIRST NAME:

(capital letters please)

MATRICOLA: SIGNATURE:

(or PERSON CODE)

- The exam is in ***written form*** and consists of ***two parts***:
- ***Theory*** (80%): Syntax and Semantics of Languages
    1. regular expressions and finite automata
    2. free grammars and pushdown automata
    3. syntax analysis and parsing methodologies
    4. language translation and semantic analysis
- ***Lab*** (20%): Compiler Design by Flex and Bison
- To pass the ***exam***, the candidate must succeed in ***both*** parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass the ***theory*** part, the candidate must sufficiently answer all ***four*** parts.

PART 1 - regular expressions and finite automata

1. Consider the ***regular expression R*** below:

$$R = (a \mid bb \mid ba)^{+}$$

Find the correct ***initials***, ***finals*** and ***digrams*** of ***R***.

Ini (R) =	Fin (R) =	Dig (R) =
Choose...	Choose...	Choose...
aa, ab, ba	aa, ab, ba	aa, ab, ba
a	a	a
a, b	a, b	a, b
aa, ab, bb	aa, ab, bb	aa, ab, bb
aa, ab, ba, bb	aa, ab, ba, bb	aa, ab, ba, bb
b	b	b
aa, ba, bb	aa, ba, bb	aa, ba, bb

2. Is the regular expression ***R*** above ***nullable*** ?

Select one:
- ○ True
- ○ False

3. Still with reference to the above regular expression ***R***, what is the ***language*** generated by its ***local sets*** ? Suitably ***justify*** your answer.

4. Is the language ***L(R)*** generated by the above regular expression **R *local*** ?

Select one:
- True
- False

5. Suitably ***justify*** your answer to the previous question.

6. Consider now the ***numbered version*** $R_{\#}$ of the above regular expression ***R***, terminated with the end-mark symbol ***t***. Notice that usually the end-mark symbol ***t*** is denoted as ⊣.

$$R_{\#} = (a_1 \mid b_2\ b_3 \mid b_4\ a_5)^+\ t$$

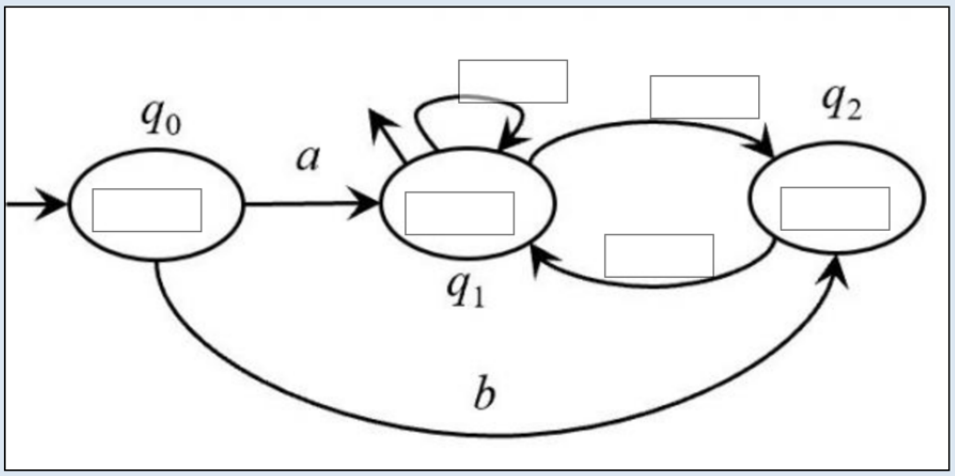
Choose the ***sets*** necessary for the construction of the ***Berry-Sethi recognizer***.

Ini ($R_{\#}$)	Fol (a_1)	Fol (b_2)	Fol (b_3)	Fol (b_4)	Fol (a_5)
Choose...	Choose...	Choose...	Choose...	Choose...	Choose...
a1 b4 t	a1 b4 t	a1 b4 t	a1 b4 t	a1 b4 t	a1 b4 t
a1 b2 b4 t	a1 b2 b4 t	a1 b2 b4 t	a1 b2 b4 t	a1 b2 b4 t	a1 b2 b4 t
a1 b2 b4	a1 b2 b4	a1 b2 b4	a1 b2 b4	a1 b2 b4	a1 b2 b4
a1 b4	a1 b4	a1 b4	a1 b4	a1 b4	a1 b4
b3	b3	b3	b3	b3	b3
a5	a5	a5	a5	a5	a5
a1 b2 t	a1 b2 t	a1 b2 t	a1 b2 t	a1 b2 t	a1 b2 t

7. For the above regexp ***R***, here reproduced:

$$R = (a \mid bb \mid ba)^{+}$$

place the correct ***initials*** and ***followers*** into the ***Berry-Sethy automaton graph*** below.



a1 b3 b4 t | b3 a5 | a, b | a1 b2 b4 | b2 a5 | a1 b3 b4 | b | a | a1 b2 b4 t

8. Is the above ***BS*** automaton ***minimal*** ?

Select one:
- True
- False

9. ***Provide*** a brief but exhaustive and clear ***justification*** to your answer to the previous question.

10. Consider the ***language*** below:

$$L((a \mid bb \mid ba)^+) \cap L(((a \mid b)^2)^*)$$

Write the first ***four strings*** of the above language in ***lexicographical*** order (as usual, consider ***a*** < ***b***).

11. Consider again the ***language*** below:

$$L((a \mid bb \mid ba)^+) \cap L(((a \mid b)^2)^*)$$

Choose the ***regular expression*** that defines it, among the following ones.

Select one:

- $((a \mid bb \mid ba)^2)^+$
- ***none*** of the other options
- $(aa \mid bb \mid ba)^*$
- $(aa \mid ab \mid ba \mid bb)^+$
- $(aa \mid bb \mid ba)^+$
- $(aa \mid bbbb \mid baba)^+$

12. ***Provide*** a suitable, brief and precise ***justification*** for your answer to the previous question.

PART 2 - Free grammars and pushdown automata

Consider a language ***L***, over the alphabet **{ *a*, "(", ")" }**, consisting of ***nested lists***, as follows:

- the depth level is arbitrary
- the list elements are denoted by letter ***a***
- the length of a (sub)list is arbitrary and a (sub)list may be empty
- and each (sub)list is opened and closed by a pair of open and closed round brackets "**(**" and "**)**", including the top level list

Here is a possible ***BNF*** grammar G_1 for language ***L*** (axiom ***S***):

S → "(" X ")"

X → ε | Y X

Y → a | "(" X ")"

Sample ***valid*** strings: $()$ (a) $(a(aa))$ $((a)aa)$ ...

13. Is grammar G_1 ***ambiguous*** ?

Select one:
- True
- False

14. Shortly ***justify*** your answer to the previous question, whether grammar G_1 is ***ambiguous*** or not.

15. Which is the ***simplest type*** of ***automaton*** that can recognize language ***L*** ?

Select one:

- ***finite*** state
- ***nondeterministic*** pushdown
- ***deterministic*** pushdown
- ***Turing*** machine

16. Try to ***simplify*** grammar G_1, and ***find*** a ***new equivalent BNF grammar*** G_2 that uses only ***two nonterminals***. Shortly ***justify*** your solution.

17. If possible, try to ***simplify*** grammar G_1 even more, and ***find another equivalent BNF grammar*** G_3 that uses only ***one nonterminal***. Shortly ***justify*** (the equivalence of) your solution, if you can find one; otherwise explain ***why*** a solution does not exist.

18. Consider the regular language generated by the regular expression ***R*** below, over the alphabet **{ *a*, "(", ")" }** of three symbols:

R = (aa | "(" | ")")*

Basing on grammar G_1, which generates language ***L***, and possibly by modifying G_1, find a ***new BNF grammar*** G_4 that generates the ***intersection*** language:

$$L \cap L(R)$$

19. Consider the ***intesection language*** below, where $\overline{L(R)}$ denotes the complement language of ***L(R)***:

$$L \cap \overline{L(R)}$$

Write the simplest possible ***EBNF*** grammar for it. Shortly ***justify*** your answer.

20. Is the ***complement language*** of the language ***L*** generated by grammar G_1, a ***context-free*** language ?

Select one:
- True
- False

21. Shortly ***justify*** your answer to the previous question, whether the complement language of language ***L*** is ***context-free*** or not.

22. This space is for writing your personal notes to answer other questions. These notes ***will not be corrected***.

23. Consider a fragment of a ***programming language*** that exhibits the following features:

- the language is a ***sequence***, possibly empty, of statements, separated by semicolon ";"
- a statement is either an ***assignment*** of an ***arithmetic expression*** to a ***scalar variable***, or a ***multiple-way conditional*** called ***select***
- an expression may contain ***n***-ary ***addition*** and ***n***-ary ***multiplication*** (with the usual operator precedence and unspecified associativity), numerical ***constants,*** scalar ***variables*** and ***subexpressions*** enclosed in round brackets
- the condition of the ***select*** structure is an expression, the ***select*** may have zero or more ***branches*** (cases), and each branch is tagged by a numerical constant
- optionally, there may be only one ***default*** branch, *not necessarily the last one*

Example (sketched):

```
alpha := beta × (gamma + delta) ;
select 2 × eta do
    branch 1 : statement list
    branch 5 : statement list
    default : statement list
    branch 2 : statement list
    ...
end select ;
...
```

Write and ***EBNF*** (i.e., extended) grammar ***G*** that generates the described programming language (axiom ***PROG***), as simple as possible; the grammar may be ambiguous. For simplicity, numerical ***constants*** and variable ***names*** can be schematized by terminals ***num*** and ***id***, respectively.

Please use the ***template*** below, with some nonterminals already provided (if necessary, add more nonterminals and their rules).

<PROG> →

<STAT> →

<ASGN> →

<SELECT> →

<EXPR> →

24. Suppose now the keyword ***end select*** is ***removed***. Which statement(s) below, related to such a modified programming language, is (are) ***true*** ?

Select one or more:

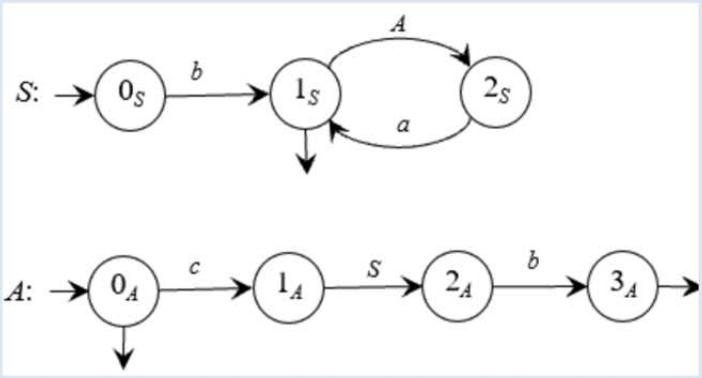
- [ ] the language has both a "dangling branch" and a "dangling default" ambiguity
- [ ] the language does not have any "dangling branch" or "dangling default" ambiguity
- [ ] the language only has a "dangling branch" ambiguity
- [ ] the language only has a "dangling default" ambiguity

25. Shortly ***justify*** the answer(s) given to the previous question, whether the modified language has any "dangling" ambiguity or not.

(ADDITIONAL SPACE FOR PREVIOUS QUESTIONS)

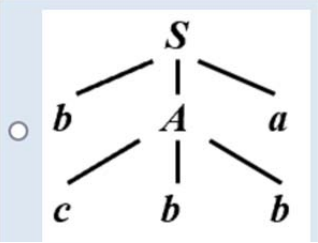
PART 3 - Syntax analysis and parsing methodologies

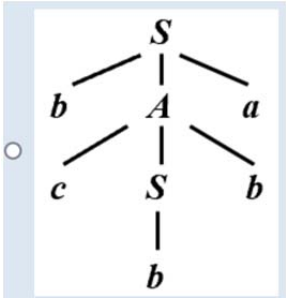
Consider the ***grammar G*** defined by the ***machine net*** below, with terminal alphabet ***a***, ***b, c***, and nonterminal alphabet ***A***, ***S*** (axiom ***S***):

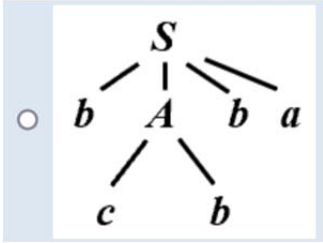


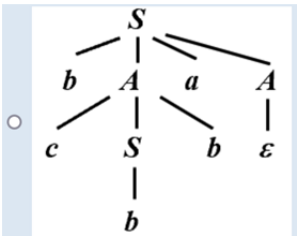
26. ***Choose*** the correct ***syntax tree*** for the string ***bcbba***.

Select one:

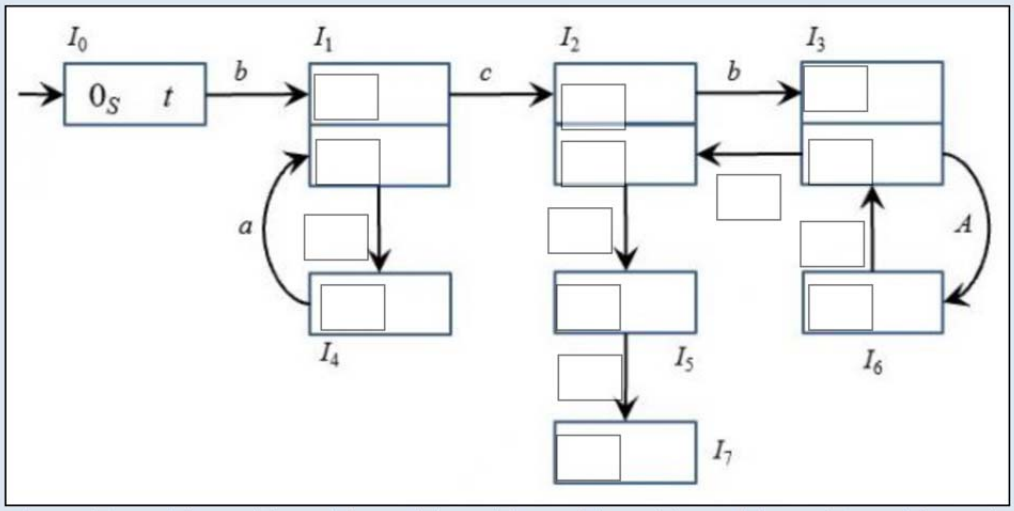








Please ***fill in*** the missing parts in the macro-states (m-states) of the following ***pilot*** automaton of the grammar ***G*** above (please notice that the letter ***t*** denotes the terminator symbol ⊣):



0S t	A	0S a	0A t	1A a	0S b	0A a	a	0A a	1A t	2S a	3A a
	2A t	S	2S t	b	c	1S b	1S a	2A a	1S t	2S b	

27.

This space is for writing your personal notes to answer the previous questions. These notes ***will not be corrected***.

28. ***Say*** if the grammar ***G*** above is of ***type ELR(1)***.

Select one:

- True
- False

For the pilot of the grammar ***G*** above, the valid string ***bcbba*** is accepted deterministically. Simulate the execution of the ***ELR syntax analyzer*** of ***G*** and list the ***stack contents*** at each move (terminal shift, nonterminal shift or reduction). Use the ***template*** table below (first row already filled in).

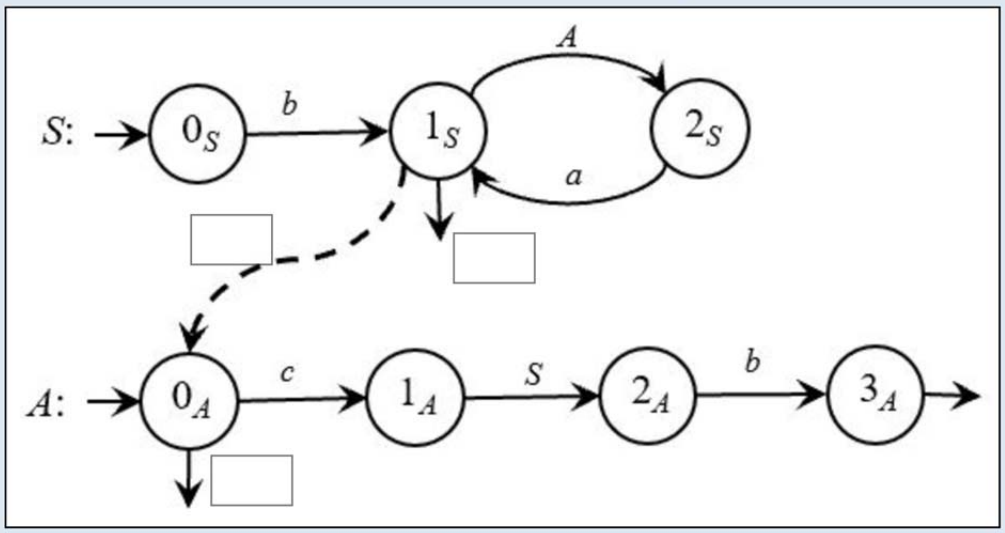
Fill the table below by overwriting the dots '...'. Number of rows not significant:

row #	stack contents ----------	input string left to analyze	description of the analyzer move
1	0	b c b b a t	initial configuration
2	0 b 1	c b b a t	terminal shift I0 → I1 on b
3	...	...	...
4	...	...	...
5	...	...	...
6	...	...	...
7	...	...	...
8	...	...	...
9	...	...	...
10	...	...	...
11	...	...	...
12	...	...	...
13	...	...	...
14	...	...	...
15	...	...	...
16	...	...	...

29.

30. This space is for writing your personal notes to answer the previous questions. These notes ***will not be corrected***.

31. Please ***fill in*** the guide ***sets on the call arcs and exit arrows*** of the machine of the grammar ***G*** above. Please note that ***t*** denotes the terminator symbol ⊣.



{a, c } | {b, t} | {a} | {a, b} | {b, c} | {a, t} | {c, t} | {b} | {c}

32. ***Say*** if the grammar ***G*** above is of ***type ELL(1)***.

Select one:
- True
- False

Simulate the execution, with input the string ***bcbba***, of the ***ELL syntax analyzer*** of ***G*** and list the ***stack contents*** at each move (shift, call or return).

Use the ***template*** table below (first row already filled in). For all the call and return moves, please indicate the relevant guide set element.

Fill in the simulation table below by overwriting the dots '...'. Number of rows not significant:

row #	stack contents ----------	input string left to analyze	description of the analyzer move
1	0S	b c b b a t	initial configuration
2	...	...	...
3	...	...	...
4	...	...	...
5	...	...	...
6	...	...	...
7	...	...	...
8	...	...	...
9	...	...	...
10	...	...	...
11	...	...	...
12	...	...	...
13	...	...	...
14	...	...	...
15	...	...	...
16	...	...	...

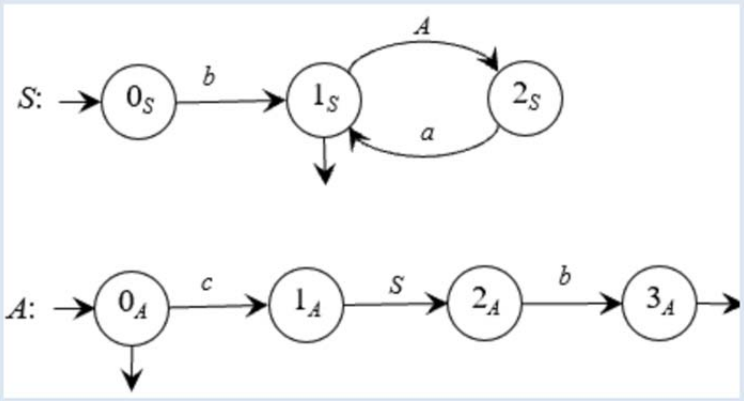
33.

34. This space is for writing your personal notes to answer the previous questions. These notes ***will not be corrected***.

35. ***Simulate*** the execution, with input the string ***bcbba***, of the ***Earley syntax analyzer*** of ***G***, and list the ***contents of the Earley vector*** **E [ ]** at each position, one row for each array element. The items inside the vector are to be written enclosed in parentheses, e.g., (**1S 4)**; you may highlight a final machine state in an item by formatting such a state in bold. Use the ***template*** below (first row already filled in).

Also ***say*** which is, in the Earley vector, the item that witnesses string acceptance.

Grammar ***G*** is reported below for your convenience.



E[0] = (0S 0)

b

E[1] = ...

c

E[2] = ...

b

E[3] = ...

b

E[4] = ...

a

E[5] = ...

The string is accepted because .....

PART 4 - Language translation and semantic analysis

The following **source *grammar* G_S** (axiom ***S***):

S → ***X S***

S → ***d***

X → ***a X***

X → ***b X***

X → ***c***

generates a language of ***two-level lists***. The top-level list is terminated by a letter ***d***, while each bottom-level list has letters ***a*** and ***b*** as its elements, and is terminated by a letter ***c***. The top level list may be empty, but its terminator is always present; the same applies to the bottom-level lists.

36. With reference to the ***source language*** defined by the grammar $\mathbf{G_S}$ above, a ***translation* τ** is defined as follows:

- in each bottom-level list, the elements ***a*** and ***b*** are ordered, first ***a*** then ***b***
- each bottom-level list is initiated by a letter ***c***, and is not terminated
- the top-level list is unchanged

Example: $\tau(abbcbacabcbaacd) = cabbcabcabcaabd$

Write a ***target grammar G*** that, combined with $\mathbf{G_S}$, provides a ***syntactic translation scheme*** that defines translation τ.

37. Is the translation τ defined before ***rational***, i.e., regular, definable through a ***rational translation*** expression ?

Select one:
- True
- False

38. Shortly ***justify*** your answer to the previous question, whether translation τ is ***rational*** or not.

39. Is the translation τ defined before ***deterministic***, i.e., the proposed scheme is of type ***(E)LL*** or ***(E)LR*** for some k ?

Select one:
- True
- False

40. Shortly ***justify*** your answer to the previous question, whether translation τ is ***deterministic*** or not.

41. Now consider the ***inverse translation*** τ^{-1}. Is it ***deterministic*** ? Shortly ***justify*** your answer.

Now, again with reference to the ***source language*** defined by the grammar G_S above, a ***new translation τ'*** is defined as follows:

- a bottom-level list is left unchanged (elements ***a*** and ***b***, and terminator ***c***), provided it contains the same numbers of elements
- otherwise, a bottom-level list is completely removed (including its terminator ***c***)
- the top-level list is unchanged (except for the removed bottom-level lists)

Example: $\tau'(abbcbacabcbaacd) = bacabcd$

Write a ***attribute grammar*** G_A that, basing on the syntactic support G_S, defines such a new ***semantic*** translation ***τ'***.

To this end, use the following ***four attributes***:

- ***na*** and ***nb*** are integer ***right*** attributes associated with ***X***, and respectively count how many elements ***a*** and ***b*** a bottom-level list contains
- ***e*** is a boolean ***left*** attribute associated with ***X*** , and tells if a bottom-level list contains the same numbers of elements ***a*** and ***b***
- ***t*** is a string ***left*** attribute associated with ***X*** and ***S***, and provides the final translation ***τ'*** in the root node (axiom ***S***) of the syntax tree

Please ***write***, in the template provided below, the ***equations*** (semantic functions) for the attributes associated with each rule of the attribute grammar G_A.

42.

1: $S_0 \rightarrow X_1\ S_2$ .....

2: $S_0 \rightarrow d$ .....

3: $X_0 \rightarrow a\ X_1$ .....

4: $X_0 \rightarrow b\ X_1$ .....

5: $X_0 \rightarrow c$ .....

43. Is the attribute grammar G_A that answers the previous question, of type ***one-sweep*** and maybe even of type ***L*** ?

Select one or more:

- ☐ ***none*** of the two options
- ☐ of type ***one-sweep***
- ☐ of type ***L***

44. Shortly ***justify*** your answer(s) to the previous question, whether the attribute grammar G_A is of type ***one-sweep*** and maybe even of type ***L***, or not.